Noname manuscript No.
(will be inserted by the editor)

Computer-Aided Generation of N-shift RWS

A Computational Approach to Generate Multi-Shift Rotational Workforce Schedules

Benjamin Bolling

Received: date / Accepted: date (awaiting publishing)

Abstract Generating schedules for shift workers is essential for many employers, whether the employer is a small or a large industrial complex, research laboratory, or other businesses involving shift works.

Previous methods for creating rotational workforce schedules included interactions between the schedule maker and the algorithm, including defining the length of sequences of consecutive days of working shifts.

In this method, an algorithm takes into account inputs (or constraints) from the schedule maker and then presents the possible solutions (incl. that all shifts must be filled, working hours per week, minimal resting time, etc.) in a first phase. The schedule maker can then select which solutions are most feasible to proceed with in the second phase, where the final schedules are then constructed and exported.

Keywords Workforce scheduling · Shift · Two-shift · Three-shift · Rotating Shift Work · Rotational Workforce · Scheduling · Schedules

1 Introduction

Creating shift work schedules has always been a challenging task, especially such that are equal for all workers and at the same time distributes the shifts evenly and properly to prevent staff burnout. In order to achieve schedules for the workers that treats everyone equally, we will focus on so-called rotational workforce schedules (RWS:s). Rotational workforce schedules means that the schedule rotates after time, and hence, the other option would be static shift schedules.

In this project, we will do some literature review of what has been done in the past to establish methods for creating RWS:s followed by a different method to create them as well as follow the examples of creating them.

B. Bolling
E-mail: benjaminbolling@icloud.com

2 Literature review

In [1], the framework proposed consists of 4 steps (as quoted from [1] p. 88):

- choosing a set of lengths of work blocks (a work block is a sequence of consecutive days of working shifts),
- choosing a particular sequence of work and blocks of days-off amongst these that have optimal weekend characteristics,
- enumerating possible shift sequences for the chosen work blocks subject to shift change constraints and bounds on sequences of shifts, and
- assignment of sequences of shifts to blocks of work while fulfilling the staffing requirements.

The framework can thus, as stated in the article, be considered as a semi-automatic way of generating schedules as the algorithm is focused on interactions with the decision-maker who has to choose a fixed set of allowable lengths for work blocks.

In [2], an algorithmic framework using a decomposition heuristic was constructed in order to quickly obtain feasible solutions for the rotational workforce scheduling problem. Here, similar to [1], the decision-maker has to set up a fixed set of work blocks. Their method is very powerful and can construct schedules for a large number of employees in a short amount of time.

In this project, we want to make the amount of interaction required less by allowing an algorithm present all possible solutions and hence giving the schedule maker all possible options for a given number of weeks to cycle over (which in this project is the same as the number of workers).

3 Computational Approach and Results

Looking back at what was done in previous work [1][2], we will use an approach similar to what was suggested in [1]: Each person has the same schedule, shifted by one week. Thus, the weeks are continuously shifted by one week for each worker until all workers have had 'each week'. This type of schedule can be created manually by a person by e.g. defining a set of work blocks ([1][2]), but in this project an algorithm has been constructed that does the work of finding possible work blocks in order to reduce the workload for generating them. In this approach, we define the term 'shift arrays' which each represent a possible schedule, similar to work blocks. It has been divided into two phases, *Boolean Shift Arrays* (in which boolean shift arrays are generated) and *From Boolean Shift Arrays to a RWS* (in which a selected boolean shift array is shaped into its final RWS layout).

3.1 Boolean Shift Arrays (phase 1)

For simplicity, we begin by using boolean shift arrays with 1 meaning that the person works and 0 meaning that the person does not work. In order to impose some constraints on the shift arrays, we define the number of working days per week as n_{wd} and number of weeks to cycle over as n_W. This results in a shift array of length $7n_W$. For constructing the different combinations of the boolean array, we use Algorithm 1.

Algorithm 1: Generation of Boolean Arrays

```
shiftarrays = [];
r = range(self.noofweeks*self.workingdays);
pool = tuple(iterable);
n = len(pool);
if r > n then
    loop = False;
else
    loop = True;
end
while loop is True do
    for i in reversed(range(r)) do
        if indices[i] != i + n - r then
            break
        end
    end
    else
        loop = False
    end
    if loop is True then
        indices[i] += 1;
        for j in range(i+1, r) do
            indices[j] = indices[j-1] + 1
        end
        shiftarray = ["0"] * (numberofweeks*workingdays);
        for ind in tuple(pool[i] for i in indices) do
            shiftarray[ind] = "1"
        end
        if shiftarray is ok then
            shiftarrays.append(shiftarray)
        end
    end
end
```

Note that the "**if** *shiftarray is ok*" in Algorithm 1 is a combination of all constraints discussed in this section and are checked before a shiftarray is allowed to be appended to the list of shiftarrays, hence ensuring that all shiftarrays in the list follow these constraints.

Since each week also resembles a worker, the shift array can be set up as a matrix with 7 columns, each representing the days of a week. The columns can then be summed to achieve the shift occupancy (or how many people are working each shift). Thus, the algorithm only allows shift arrays to pass for which all shifts are occupied by at least one worker, with a shift represented by the first n_{wd} days for each week. In order to extend to not only use single shifts but also 2- or 3-shifts, a simple logical reasoning was added into the algorithm. For N shifts per day, each day has to be filled with at least N workers.

The next input that the algorithm needs is the shift lengths and the weekly working hours per worker, defined as t_s and t_W, respectively. However, in order

to generate "good schedules", an additional constraint will be needed in order to avoid all working days being clustered together.

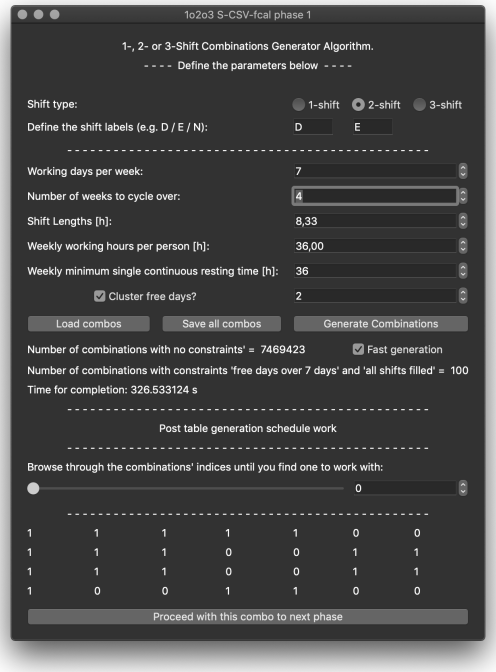

Fig. 1 The RWS:ing Application's algorithm's "phase 1 GUI".

Hence, the constraint for weekly minimum single continuous resting time is added, defined as t_r. The algorithm ensures that all passed shift arrays have at least this many hours of free-time for each week. The number of shifts n_S per shift array is calculated by

$$n_S = \text{ceil}\left(\frac{t_W}{t_s}\right).$$

The reason for using ceiling function and not the floor function is simply the argument that it is better with a couple of more hours than fewer. In order to cluster days off (n_cf), the algorithm's GUI has an optional additional constraint that serves this purpose and simply does not allow shift arrays with 0:s in clusters less than this through.

By using the input $n_W n_{wd}$ as an iterable and n_S as the length of subsequences of elements from the iterable, we use the same methodology as *itertools* [3] module in Python to create each shift array. The other inputs are used as constraints on whether the shift array should be appended to the array of shift arrays or trashed. The reasoning for not using the built-in module itertools.combinations [3] is that it returns all array combinations it could find. Without the constraints, the returned arrays become too large for a normal up-to-date computer's internal memory to handle.

With this, the final result is an array of shift arrays in which each shift array is filled with $7n_S$ 1:s and $n_W(7 - n_S)$ 0:s whilst obeying the above mentioned constraints. The number of possible combinations C using Algorithm 1 can be expressed as:

$$C = \frac{(n_W n_{wd})!}{n_S!(n_W n_{wd} - n_S)!}. \tag{1}$$

Table 1 Parameters selected for the generation of a N-shift RWS.

N	n_{wd}	n_W	t_S	t_W	t_r	n_cf	Shift types' labels
2	7	4	8.33	36.00	36	2	D, E

The parameters selected for the RWS is defined in Table 1. These values are then reflected in the algorithm's GUI for phase 1 can be seen in Figure 1. Note that the generated shift arrays can be browsed through using the slider or the numerical input field and that the shift arrays are constructed such that each week or worker (depending on the viewing angle) is represented in separate rows.

3.2 From Boolean Shift Arrays to RWS (phase 2)

Fig. 2 The RWS:ing Application's algorithm's "phase 2 GUI" as launched from the "phase 1 GUI" and with the first Thursday's shift changed to an evening shift (left) and after some simple shift type permutations (right).

In this phase, we have chosen to proceed with combination #35741 since it has two out of four weekends off (note the zeroes in the bottom table in Figure 1). By proceeding, the "phase 2 GUI" is launched with the selected array as input as can be seen in Figure 2 to the left.

The free days are all represented by zeroes whilst all other shifts (ones) are converted to the first defined shift type label. For $N > 1$, each shift can be replaced by another shift via dropdown menus. The GUI shows the number of shifts of each type each week or worker has and a table with the results, i.e. number of worker per shift and day. Shifts that are occupied have green background whilst shifts that are unoccupied have a red background.

If the continuous resting time between two assigned shifts is too low, the background colour of the second shift becomes red (e.g. a Friday day-shift after a Thursday evening-shift if the continuous resting time has to be at least 11 hours, as shown in Figure 2 to the left). With enough resting time in between shifts, the background of the second shift is be green.

After some simple shift types permutations, we end up with what is shown in Figure 2 (right). If the schedule is deemed as finished, the export button allows for the export of the finalized table into a txt- or CSV-file.

It is also possible to combine two or more shift arrays, e.g. two 4-week shift cycles to achieve a single 8-week shift cycle and which could result in that each shift is occupied by 2 workers. Such an example is shown in Figure 3, in which we have generated Boolean Arrays and set that free days have to be clustered in clusters of two (minimum). We then combined two 4-week shift cycles (#81 and #210), worked on them in phase 2, exported them as CSV-files and finally imported them into Microsoft Excel (2018) (where colouring work was done to highlight the weeks).

Fig. 3 Two 4-week shift cycles with free days clustered (minimum 2) and being worked on in phase 2 (top figures), exported to CSV-files and then imported into a single spreadsheet within Microsoft Excel (2018).

4 Benchmarking results

The algorithm benchmarking was done on an Apple MacBook Pro with the specifications as defined in Table 2.

Table 2 Benchmarking computer specifications.

Computer type:	Apple MacBook Pro (13-inch, 2019)
OS:	macOS Mojave v. 10.14.6
Processor:	2.8 GHz Intel Core i7 processor
Internal Memory:	16 GB 2133 MHz LPDDR3
Graphics Card:	Intel Iris Plus Graphics 655 1536 MB

In the GUI, there is a "fast generation" checkbox which stops the algorithm from further calculations once the first 100 solutions have been found. This way, computation time can be lowered (in comparison to "full generation" which will go through all possible solutions from the boolean array). For our example, the time it took to complete decreased from 508.7 s (for a full generation) to 24.55 s (for the full generation) (see Table 3), which is a decrease in time by 95%.

We use the parameters defined in Table 1, with the exception of N and Shift types' labels. Note that for Table 3 and Table 4, the number (#) of weeks given is the minimum amount of weeks required for a full shift cycle in order to find solutions for the N-shift problems (with $N = 1, 2, 3$ for single-, two- and three-shifts, respectively). The free days clustering option is not selected for the benchmarking.

Table 3 Benchmarking for fast and full generation of the Boolean Arrays (as defined in Section 3.1 for Phase 1).

Type:	# of weeks:	Time (fast) [s]:	Time (full) [s]:
Single-shift, 5 days/week	1	7.224e-05	7.224e-05
Single-shift, 7 days/week	2	1.497e-02	5.211e-02
Two-shift, 7 days/week	4	24.55	508.7
Three-shift, 7 days/week	5	3 087	6.627e+04

Table 4 Number of combinations and solutions found for full generations of the Boolean Arrays (as defined in Section 3.1 for Phase 1).

Type:	# of weeks:	Combinations:	Solutions:
Single-shift, 5 days/week	1	1	1
Single-shift, 7 days/week	2	2 002	462
Two-shift, 7 days/week	4	13 123 110	1 668 226
Three-shift, 7 days/week	5	1 476 337 800	11 383 225

Plotting the benchmarking results, we find the logarithmic graph given in Figure 4. As can be seen, the computation time T_C increases exponentially with the

number of weeks in a shift cycle on average in accordance with

$$T_C(full) = \exp(5.046 \times n_W) \times 9 \times 10^{-7}$$

and

$$T_C(fast) = \exp(4.254 \times n_W) \times 2 \times 10^{-6}$$

for the full and fast generations on, respectively.

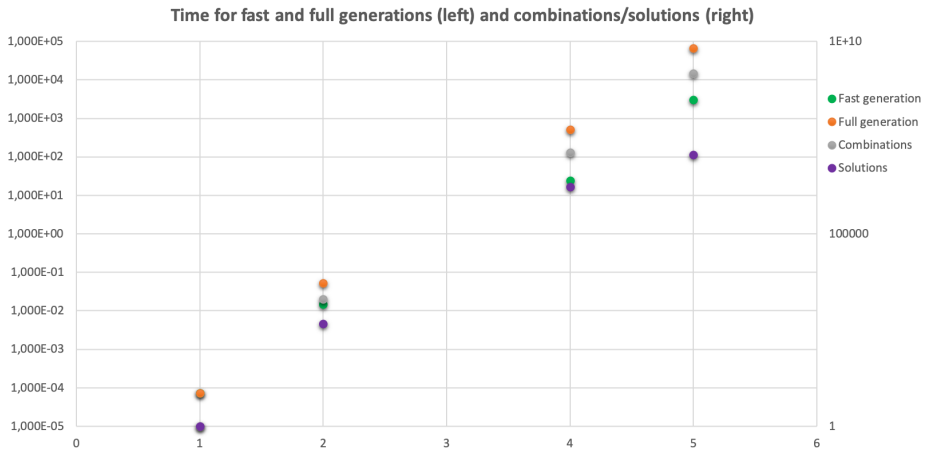

Fig. 4 The benchmarking results in respect of time for fast- and full generation of the boolean arrays (on the left vertical axis), and the number of combinations gone through and the solutions found (on the right vertical axis).

5 Conclusions

In this article, we have demonstrated that the constructed algorithm can generate schedules for different number of weeks to cycle over. The current issue is that the computational complexity (and hence the computation time) increases with the number of weeks, as can be seen in Table 4 and Figure 4. This means that for a higher amount of weeks in a shift cycle, this application will need development in order to have more efficient ways of finding the solutions and/or deployment of the application onto super-computers for generating the Boolean Arrays.

For up to 5 weeks in a shift cycle, it is, however, possible to use a general-purpose computer such as the benchmarking MacBook Pro with specifications defined in Table 2. It has thus been demonstrated that the application can be used to generate 1, 2 and 3-shift schedules. Future development plans include adding an automated assignment function of shift types in phase 2, which would further strengthen the usability of this application.

References

1. N. Musliu, J. Gärtner, and W. Slany, Efficient generation of rotating workforce schedules, Discrete Applied Mathematics, Vol. 118, pp. 85-98 (Issues 1–2, 15 April 2002).
2. T. Becker, A decomposition heuristic for rotational workforce scheduling, Journal of Scheduling (2020).
3. G. Van Rossum, The Python Library Reference, release 3.8.2, Python Software Foundation (2020).